Week 11 — Model Compression Benchmark

Course: DS5619 Machine Learning Systems Operations
Student ID: `142301024`
Seed: `846403649` (salt: `week11`)

What Was Implemented

In this lab, I implemented model compression techniques (magnitude-based pruning and symmetric INT8 quantization) for a 256-parameter linear classifier head in `src/compression.py`.

The five functions implemented are:

1. **`model_size_bytes(weights, dtype_bytes)`**:
   - Calculates the dense storage size in bytes: `len(weights) * dtype_bytes` (e.g. 4 bytes for FP32, 1 byte for INT8).
   - Returns 0 if the weights list is empty.
2. **`prune_weights(weights, sparsity)`**:
   - Computes how many weights to zero out: `n_prune = int(sparsity * len(weights))`.
   - Sorts indices by absolute weight magnitude `abs(w)` and identifies the smallest `n_prune` weights.
   - Returns a new list with those indices set to `0.0`, leaving other weights unchanged and without mutating the original input list.
3. **`sparse_coo_size_bytes(weights, value_bytes=4, index_bytes=4)`**:
   - Calculates the storage needed if non-zero weights are stored in sparse Coordinate (COO) format.
   - Counts surviving non-zero weights (`w != 0.0`) and multiplies by `(value_bytes + index_bytes)` (8 bytes each: 4 for the float value + 4 for the index). Returns 0 for all-zero or empty inputs.
4. **`quantize_int8(weights)`**:
   - Symmetric linear quantization mapping floats to signed 8-bit integers $[-127, 127]$.
   - Calculates scale: `scale = max(abs(w)) / 127.0` (defaults to `1.0` if all weights are zero).
   - Rounds each weight: `round(w / scale)` and clamps it to $[-127, 127]$.
   - The largest magnitude weight maps to exactly ± 127, and round-trip error through `dequantize_int8` is bounded by at most half a quantization step (`scale / 2`).
5. **`benchmark_model(label, weights, eval_examples, dtype_bytes, n_latency_reps=50)`**:
   - Calculates storage size using `model_size_bytes`.
   - Evaluates classification accuracy over 300 test examples, rounded to 4 decimal places.
   - Measures wall-clock execution time over `n_latency_reps` complete passes using `time.perf_counter()`, averaging the time to milliseconds per pass.

Results

Running the pipeline (`python src/run_pipeline.py`) benchmarked the four variants and produced `compression_report.json`:

Model Variant	Storage Format	Size	Compression Ratio	Accuracy	Latency (ms/pass)
`baseline_fp32`	Dense FP32	1024 B	1.00×	100.00%	~2.77 ms
`pruned_dense_fp32`	Dense FP32 (70% zeros)	1024 B	1.00× *(pitfall)*	99.67%	~2.93 ms
`pruned_sparse_coo`	Sparse COO (77 survivors)	616 B	1.66×	99.67%	~2.93 ms
`quantized_int8`	Dense INT8	256 B	4.00×	99.67%	~2.76 ms

Key Observations:

1. **The Pruning Pitfall (`pruned_dense_fp32` vs `pruned_sparse_coo`):**
   - Both pruned models have the exact same weights and identical accuracy (99.67%).
   - But in dense storage, zeros still take 4 bytes each, so `pruned_dense_fp32` is still 1024 bytes (0 bytes saved).
   - In sparse COO, zeros are omitted, but each survivor pays 8 bytes (value + index). With 77 survivors, size drops to $77 \times 8 = 616$ bytes (1.66× compression).
   - Sparse COO only beats dense FP32 when sparsity is $> 50\%$ (where the saved zeros outweigh the 4-byte index tax on survivors).
2. **Quantization Wins on Footprint:**
   - INT8 quantization achieves an exact 4.00× compression (256 bytes) with almost zero accuracy drop (99.67%), without needing any index overhead.
3. **The 0.95 Sparsity Cliff:**
   - The weights have 40 informative weights ($| w | \geq 0.50$) and 216 noise weights ($| w | \leq 0.03$).
   - At 70% sparsity, 179 weights are pruned. Since $179 < 216$, only noise weights are zeroed, keeping accuracy at 99.67%.
   - At 95% sparsity, 243 weights are pruned. This exhausts all 216 noise weights and cuts into the signal, wiping out 27 informative weights. With only 13 informative weights left, accuracy drops off a cliff to **79.33%**.